

IMDA GCP DEPLOYMENT: STEP-BY-STEP PARTNER GUIDE

High-Volume SAP Document AI Pipeline (docai_sap_pipeline) — 25,000 Pages/Week Automated Invoice & Purchase Order Extraction into BigQuery

DIRECTORY: docai_sap_pipeline/ | WORKLOAD: 25,000 PAGES/WK (~950 SGD/WK DOCAI) | 100% ENVIRONMENT-VARIABLE DRIVEN

Version: 2.0 (Production Distribution Release — docai_sap_pipeline Directory Edition)
Target Audience: Google Cloud Partners, Customer Engineers, SAP Enterprise Architects, and DevOps Teams

1. Executive Summary & docai_sap_pipeline Directory Inventory

This step-by-step deployment guide walks you through setting up and running the `docai_sap_pipeline/` workload in your own Google Cloud environment.

The pipeline automates the extraction of SAP Vendor Invoices (**MIRO / VBRK**) and Customer Purchase Orders (**VA01 / VBAK**) at scale (**25,000 pages/week** across 5,000 multi-page PDF documents, driving **~950 SGD/week** in Document AI SKU consumption). It automatically provisions **Enterprise Invoice Parser (INVOICE_PROCESSOR)** and **Custom/Form Document Extractor (FORM_PARSER_PROCESSOR)**, processes PDFs via 5 concurrent asynchronous batch jobs (**BatchProcessRequest**), and streams structured SAP header and line-item records (**LIFNR, KUNNR, MATNR, NETWR, MWSKZ, WAERK**) directly into **BigQuery**.

Complete File Inventory of docai_sap_pipeline/

File / Directory	Purpose & Role in Deployment
<code>01_generate_sample_sap_pdfs.py</code>	Generates realistic multi-page SAP Vendor Invoices (INVOICE) and Customer Purchase Orders (PURCHASE_ORDER) inside <code>generated_pdfs/</code> (supports 30-page test batches and 25,000-page weekly production workloads).
<code>02_setup_and_deploy_gcp.sh</code>	Provisions all Google Cloud infrastructure: enables APIs, creates GCS input/output buckets, creates the BigQuery dataset & table, provisions both Document AI processors via REST API, and writes <code>gcp_docai_config.env</code> .
<code>03_run_docai_batch_processor.py</code>	Orchestrates concurrent asynchronous Document AI BatchProcessRequest jobs (in safe 1,000-document chunks) and streams extracted SAP entities into BigQuery (<code>sap_docai_staging.extracted_sap_documents</code>).
<code>04_deploy_cloud_run_job.sh</code>	Builds the container image via Cloud Build (Dockerfile), deploys a serverless Cloud Run Job , and configures Cloud Scheduler to execute the pipeline daily at 02:00 AM SGT .
<code>run_all_25000_pages_one-shot.sh</code>	One-shot master script that provisions GCP resources, generates all 5,000 PDFs (25,000 pages), uploads them via multi-threaded <code>gcloud storage rsync</code> , and launches all 5 batch jobs concurrently in parallel.
<code>gcp_docai_config.env</code>	Environment configuration file storing your active GCP_PROJECT_ID , bucket URIs, BigQuery dataset/table names, and generated Document AI Processor IDs.

File / Directory	Purpose & Role in Deployment
Dockerfile	Production container definition (python:3.11-slim) packaging 01_generate_sample_sap_pdfs.py and 03_run_docai_batch_processor.py for Cloud Run Jobs.
README.md	Quick-reference documentation and architecture diagram for the docai_sap_pipeline/ folder.
generated_pdfs/	Folder containing the generated multi-page SAP Invoice and Purchase Order PDF documents ready for GCS upload.
sample_bigquery_extracted_rows.jsonl	Local dry-run output file generated by 03_run_docai_batch_processor.py --mode local-verify for inspecting the BigQuery row schema before live GCP execution.

2. Environment Configuration (Set Once & Copy-Paste Everything)

All scripts inside **docai_sap_pipeline/** are **100% environment-variable driven** with zero hardcoded personal credentials or project IDs.

Environment Variable	Description	Default / Example Value
GCP_PROJECT_ID	Target Google Cloud Project ID where billing is enabled	acme-sap-ai-prod
GCP_ACCOUNT_EMAIL	Google Cloud user or admin email with deployment permissions	admin@acme-corp.com
GCP_DOCAI_LOCATION	Document AI multi-region location (us or eu only)	us
GCP_BQ_LOCATION	BigQuery & GCS bucket multi-region (US , EU , or asia-southeast1)	US
GCP_CLOUD_RUN_REGION	Cloud Run & Cloud Scheduler region for automated daily jobs	asia-southeast1

Export Your Environment Variables Now

Run this block in your terminal once (replacing **YOUR_GCP_PROJECT_ID** and **YOUR_GCP_ACCOUNT_EMAIL** with your own values). Every command in the rest of this guide will automatically use these variables:

```
export GCP_PROJECT_ID="YOUR_GCP_PROJECT_ID"
export GCP_ACCOUNT_EMAIL="YOUR_GCP_ACCOUNT_EMAIL"
export GCP_DOCAI_LOCATION="us"
export GCP_BQ_LOCATION="US"
export GCP_CLOUD_RUN_REGION="asia-southeast1"
```

Required Google Cloud IAM Roles

Ensure **\${GCP_ACCOUNT_EMAIL}** has either **Project Owner / Editor (roles/owner / roles/editor)** or the following granular IAM roles on **\${GCP_PROJECT_ID}**:

- **roles/serviceusage.serviceUsageAdmin** (to enable Google Cloud APIs)
- **roles/documentai.admin** (to create and execute Document AI processors)
- **roles/storage.admin** (to create GCS input/output buckets and upload PDFs)
- **roles/bigquery.admin** (to create datasets and insert extracted SAP records)

- **roles/run.admin**, **roles/cloudbuild.builds.editor** & **roles/cloudscheduler.admin** (to deploy scheduled Cloud Run jobs)

3. Prerequisites & Workstation Setup

Step 3.1: Install Google Cloud SDK (gcloud & bq)

On macOS (via Homebrew):

```
brew update && brew install --cask google-cloud-sdk
```

Add the SDK to your shell profile (`~/.zshrc` or `~/.bash_profile`) and disable corporate mTLS client-certificate proxy conflicts:

```
echo 'export PATH="/opt/homebrew/bin:/opt/homebrew/share/google-cloud-sdk/bin:$PATH"' >>
~/.zshrc
echo 'export CLOUDSDK_CONTEXT_AWARE_USE_CLIENT_CERTIFICATE=false' >> ~/.zshrc
source ~/.zshrc
```

On Linux / Cloud Shell / Windows WSL:

```
curl https://sdk.cloud.google.com | bash
exec -l $SHELL
export CLOUDSDK_CONTEXT_AWARE_USE_CLIENT_CERTIFICATE=false
```

Verify installation:

```
gcloud version
```

Step 3.2: Install Python Dependencies

Ensure Python 3.10+ is installed, then install the required Google Cloud client libraries:

```
pip3 install --upgrade \
google-cloud-documentai \
google-cloud-storage \
google-cloud-bigquery
```

4. Step-by-Step Authentication with Your GCP Credentials

Step 4.1: Authenticate with Google Cloud CLI & Application Default Credentials

Run the following commands to sign in with your Google Cloud account (`${GCP_ACCOUNT_EMAIL}`):

```
gcloud auth login "${GCP_ACCOUNT_EMAIL}"
gcloud auth application-default login
```

Note: A browser window will open. Sign in with your Google Cloud credentials and click **Allow**. If running on a remote headless server, add **--no-launch-browser** and copy the verification URL into your local browser.

Step 4.2: Set Your Active Google Cloud Project

```
gcloud config set project "${GCP_PROJECT_ID}"
```

Step 4.3: Verify Active Account and Project

```
gcloud auth list
gcloud config get-value project
```

Confirm that the output displays an asterisk (*) next to **\${GCP_ACCOUNT_EMAIL}** and prints **\${GCP_PROJECT_ID}**.

5. Step-by-Step Deployment of docai_sap_pipeline (25,000 Pages/Week)

Navigate into the **docai_sap_pipeline** directory:

```
cd docai_sap_pipeline
```

Option 1: One-Shot Master Deployment (run_all_25000_pages_oneshot.sh)

To provision all Google Cloud infrastructure, generate 5,000 multi-page SAP PDF documents (**25,000 pages total**), upload them via multi-threaded parallel sync, and execute all **5 x 1,000-document Document AI Batch Jobs concurrently in parallel**, run a single command:

```
./run_all_25000_pages_oneshot.sh "${GCP_PROJECT_ID}" "${GCP_DOCAI_LOCATION}"
```

Option 2: Granular Step-by-Step Execution (Individual Scripts)

If you prefer to execute and verify each script inside **docai_sap_pipeline/** step by step, follow Steps 5.1 through 5.5 below:

Step 5.1: Generate SAP Invoices & Purchase Order PDFs (01_generate_sample_sap_pdfs.py)

- **Quick Test Batch (10 PDFs = 30 pages):**

```
python3 01_generate_sample_sap_pdfs.py --total-pages 30 --pages-per-doc 3
```

- **Full Production Weekly Workload (5,000 PDFs = 25,000 pages):**

```
python3 01_generate_sample_sap_pdfs.py --total-pages 25000 --pages-per-doc 5
```

- **Optional Local Dry-Run Verification (No GCP API Calls):**

```
python3 03_run_docai_batch_processor.py --mode local-verify
```

This verifies all generated PDFs locally and writes the sample BigQuery payload to **sample_bigquery_extracted_rows.jsonl**.

Step 5.2: Provision Google Cloud Infrastructure & Document AI Processors (02_setup_and_deploy_gcp.sh)

Run the automated provisioning script using your exported environment variables:

```
./02_setup_and_deploy_gcp.sh "${GCP_PROJECT_ID}" "${GCP_DOCAI_LOCATION}"  
"${GCP_BQ_LOCATION}"
```

What **02_setup_and_deploy_gcp.sh** automatically creates in your GCP project:

1. Enables APIs: **documentai.googleapis.com**, **storage.googleapis.com**, **bigquery.googleapis.com**, **run.googleapis.com**, **cloudscheduler.googleapis.com**.
2. Creates Cloud Storage Buckets:
 - Input Bucket: **gs://\${GCP_PROJECT_ID}-sap-docai-input**
 - Output Bucket: **gs://\${GCP_PROJECT_ID}-sap-docai-output**
3. Creates BigQuery Dataset & Staging Table:
 - **\${GCP_PROJECT_ID}.sap_docai_staging.extracted_sap_documents**
4. Calls the Document AI REST API to create two processors:
 - **sap-enterprise-invoice-parser** (**INVOICE_PROCESSOR**)
 - **sap-po-custom-extractor** (**FORM_PARSER_PROCESSOR**)
5. Automatically populates your generated Processor IDs and bucket paths into **gcp_docai_config.env**.

Step 5.3: Sync All Generated PDFs to Google Cloud Storage

Load **gcp_docai_config.env** and use multi-threaded **gcloud storage rsync** to upload all PDFs in **generated_pdfs/** to your input bucket:

```
source gcp_docai_config.env  
gcloud storage rsync generated_pdfs "${GCP_INPUT_BUCKET}/incoming"  
--project="${GCP_PROJECT_ID}"
```

Step 5.4: Execute Concurrent Document AI Batch Processing (03_run_docai_batch_processor.py)

Run the batch processor script in **gcp-batch** mode. It reads **gcp_docai_config.env**, chunks the 5,000 PDFs into safe batches of 1,000 documents (respecting Google Cloud's 5,000-document limit per request), fires all batch requests **concurrently in parallel**, and streams extracted SAP fields into BigQuery:

```
python3 03_run_docai_batch_processor.py \  
--mode gcp-batch \  
--project-id "${GCP_PROJECT_ID}" \  
--location "${GCP_DOCAI_LOCATION}" \  
--batch-size 1000 \  
--process-all
```

Step 5.5: Verify Extracted SAP Data in BigQuery

Query your BigQuery staging table to inspect the extracted SAP Invoices and Purchase Orders:

```
bq query --use_legacy_sql=false \  
"SELECT document_id, document_type, sap_document_number, sap_vendor_lifnr,  
       sap_customer_kunnr, currency, net_amount, tax_amount, total_amount, confidence_score  
FROM \`${GCP_PROJECT_ID}.sap_docai_staging.extracted_sap_documents\  
ORDER BY processed_timestamp DESC  
LIMIT 15"
```

6. Deploying Automated Daily Production Schedule (04_deploy_cloud_run_job.sh & Dockerfile)

To run this pipeline automatically every day at **02:00 AM SGT** without manual intervention (processing ~3,570 pages/day = 25,000 pages/week):

```
./04_deploy_cloud_run_job.sh "${GCP_CLOUD_RUN_REGION}"
```

What **04_deploy_cloud_run_job.sh** deploys:

1. Packages **Dockerfile**, **01_generate_sample_sap_pdfs.py**, and **03_run_docai_batch_processor.py** into a container image via Google Cloud Build (**gcr.io/\${GCP_PROJECT_ID}/sap-docai-batch-processor-job:latest**).
2. Deploys a serverless **Google Cloud Run Job (sap-docai-batch-processor-job)** configured with your **gcp_docai_config.env** variables (**2Gi** memory, **3600s** timeout).
3. Creates a **Google Cloud Scheduler** HTTP trigger (**sap-docai-batch-processor-job-daily-trigger** on schedule **0 2 * * * Asia/Singapore**) that executes the Cloud Run Job daily.

To trigger an immediate test run of the deployed Cloud Run Job on Google Cloud:

```
gcloud run jobs execute sap-docai-batch-processor-job \  
--region="${GCP_CLOUD_RUN_REGION}" \  
--project="${GCP_PROJECT_ID}"
```

7. Troubleshooting & Partner FAQ

Q1: ERROR: (gcloud.auth.login) Enterprise certificate provider (ECP) binary path was not found

Cause: Corporate workstations often set **CLOUDSDK_CONTEXT_AWARE_USE_CLIENT_CERTIFICATE=true** globally, causing **gcloud** to look for an mTLS certificate helper binary.

Fix: Run:

```
export CLOUDSDK_CONTEXT_AWARE_USE_CLIENT_CERTIFICATE=false  
gcloud config set context_aware/use_client_certificate false
```

Q2: Number of input documents specified (5010) exceeds the maximum supported (5000)

Cause: Google Cloud Document AI enforces a hard limit of 5,000 documents per single **BatchProcessRequest**.

Fix: Always run **03_run_docai_batch_processor.py** with **--batch-size 1000 --process-all**. It automatically splits any number of documents in your GCS bucket into safe 1,000-document chunks and submits them concurrently in parallel.

Q3: How do I customize the PDFs with my customer's actual SAP Vendor/Customer Master codes?

Edit the **SAP_VENDORS**, **SAP_CUSTOMERS**, and **SAP_MATERIALS** arrays at the top of

01_generate_sample_sap_pdfs.py to match your customer's exact SAP **KNA1**, **LFA1**, and **MARA** master records, then re-run **01_generate_sample_sap_pdfs.py**.